

NanoFlow

Towards Optimal Large Language
Model Serving Throughput

2025.09.15 이유찬

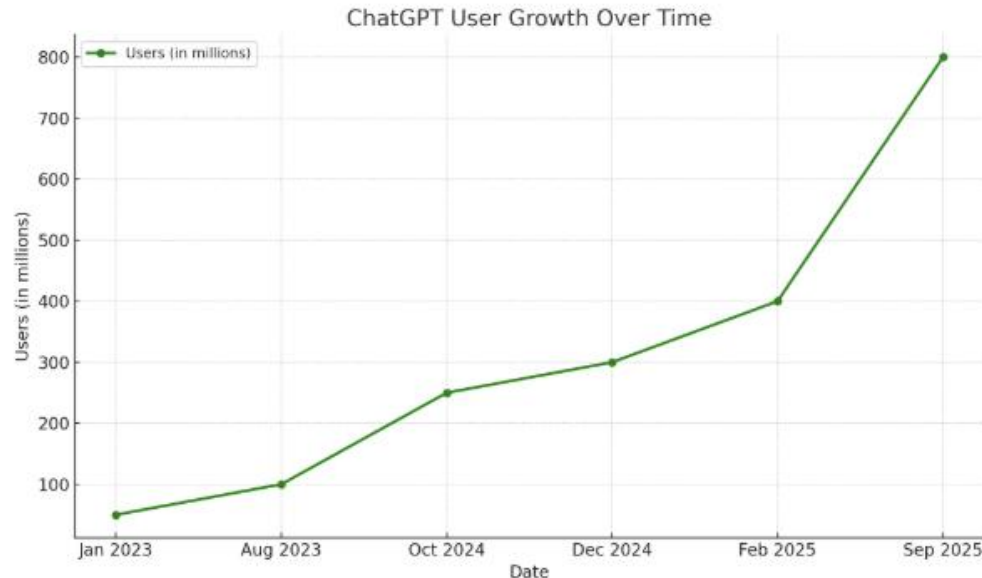


Paper info

- ▶ OSDI '25
- ▶ Kan Zhu, Yufei Gao, Yilong Zhao et al.
- ▶ University of Washington, Tsinghua University, University of Michigan, UC Berkeley

Problem

- ▶ 모델 크기, 데이터 크기, 수요 모두 증가
- ▶ GPU를 효율적으로 사용하는 것이 중요
 - ▶ 현재 방식으로는 GPU FLOPS에 비해 절반도 사용 못 함
- ▶ 대부분의 병목은 compute 자원에서 발생



Background

► Heterogeneous operations

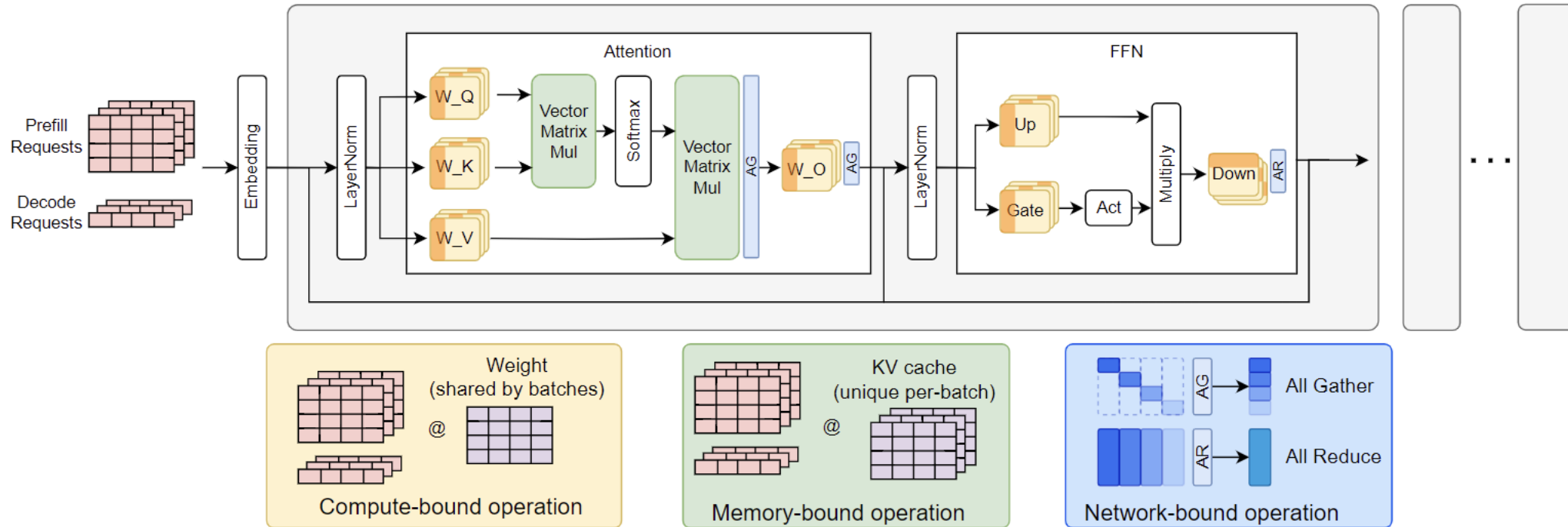


Figure 1: Transformer architecture. The operations in the yellow boxes have large batch sizes and share model weight parameters across requests; hence, they are compute-bound. Operations in green boxes require loading a unique KV cache for each request; hence, they are memory-bound. The blue box represents network operations that perform synchronization between operations.

Analysis

Contents

- ▶ Cost model of LLM Serving
- ▶ Classify LLM Serving according to Cost model
- ▶ Empirically validate
- ▶ Optimal serving throughput
- ▶ Compare with existing systems
- ▶ Intra-device parallelism

Key Factors of Serving Throughput

Hardware specification. The following factors determine the available GPU hardware resources:

- N_{GPU} : number of GPUs
- $MemBW$ (GB/s): aggregate GPU memory bandwidth
- $MemSize$ (GB): aggregate GPU memory capacity
- $Compute$ (GFLOP/s): aggregate GPU compute capacity
- $NetBW$ (GB/s): aggregate GPU interconnect bandwidth

User query statistics. The following user input statistics also affect the system throughput:

- p : average number of tokens in prompts to be prefilled
- d : average number of tokens in output to be decoded

Model configuration. The following factors regarding the model architecture determine the computation, memory, and network demand when serving LLMs:

- D_{model} : hidden dimension size
- L : number of layers
- P_{Model} : number of parameters
- R_{GQA} : group size of GQA ¹
- S_{type} (Bytes): number of bytes (size) of the data type for model parameters (e.g., 2 for FP16)

Cost Model of LLM Serving

▶ Memory

Memory. From the memory perspective, latency is:

$$T_{mem} = \frac{MemSize}{MemBW} \quad (1)$$

▶ Compute

▶ Dense operation에서 GEMM은 $2B_{Dense} \sum N_w K_w$. $T_{Compute} \approx \frac{2B_{Dense} \cdot P_{Model}}{Compute}$

▶ Network

$$T_{net} \approx 4 \cdot \frac{(N_{GPU} - 1) B_{Dense} D_{model} S_{type} L}{NetBW} \quad (3)$$

Classification of LLM Serving Workloads

$$\frac{T_{Net}}{T_{Compute}} = \frac{2D_{model}L}{P_{model}} \frac{(N_{GPU} - 1)Compute}{NetBW / S_{type}}$$

$$T_R = \frac{T_{Mem}}{T_{Compute}} \approx \frac{Compute}{MemBW} \frac{MemSize}{P_{model}} \frac{1}{2B_{dense}}$$

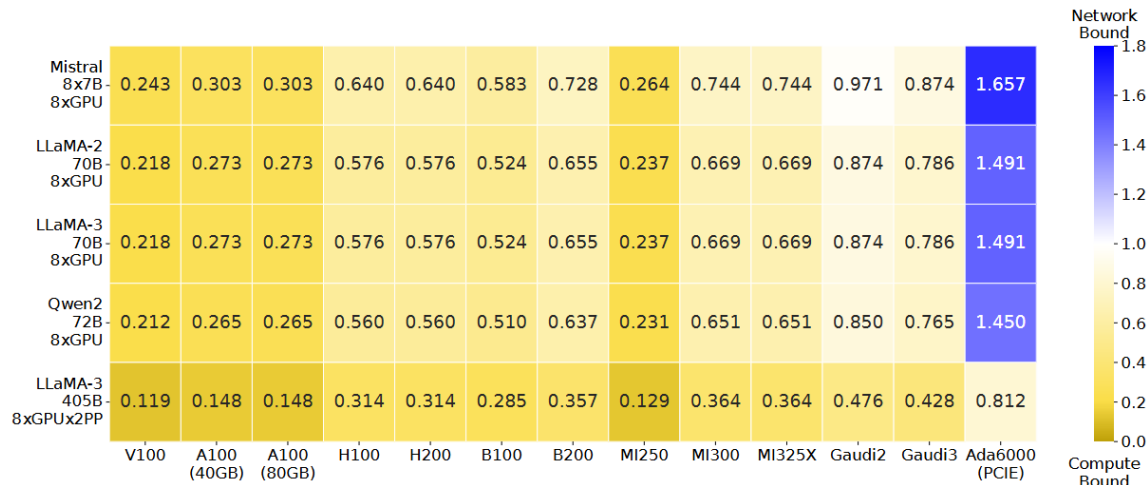


Figure 2: Comparison of network time and compute time. The closer to yellow, the more compute-bound the workload is, whereas the closer to blue indicates the workload is more network-bound.

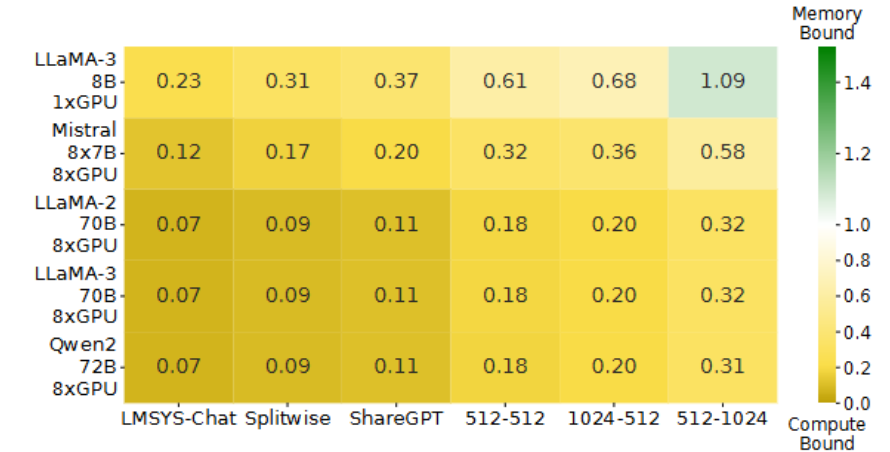


Figure 3: Comparison of compute time and memory time. The closer to yellow, the more compute-bound the workload is, whereas the closer to green the more memory-bound it becomes.

Validation of the Cost Model

Table 2: Comparison of operation runtimes between cost model estimation and real-world measurements.

Operation	Compute (GFLOP)	Mem Load (GB)	Net Usage (GB)	Est. T_{comp} (ms)	Est. T_{mem} (ms)	Est. T_{net} (ms)	Real Time (ms)
KQV	27487.8	19.5	0	11.01	1.22	0	16.08
O	21990.2	16.1	0	8.81	1.01	0	16.01
UG	153931.6	96.6	0	61.67	6.04	0	69.92
D	76965.8	49.7	0	30.84	3.11	0	34.96
DecAttn	3665.9	462.2	0	1.47	28.89	0	35.60
PfAttn	916.3	2.1	0	0.37	0.13	0	4.56
Net	18.8	75.2	75.2	0.01	4.70	31.33	47.92
Total				114.17	45.09	31.33	

Optimal Serving Throughput & Gap to Optimal Throughput

► Optimal Serving Throughput

$$\text{Throughput}_{\text{optimal}} = \frac{B_{\text{Dense}}}{T_{\text{Compute}}} = \frac{\text{Compute}}{2P_{\text{Model}}} \quad (5)$$

► Gap to Optimal Throughput

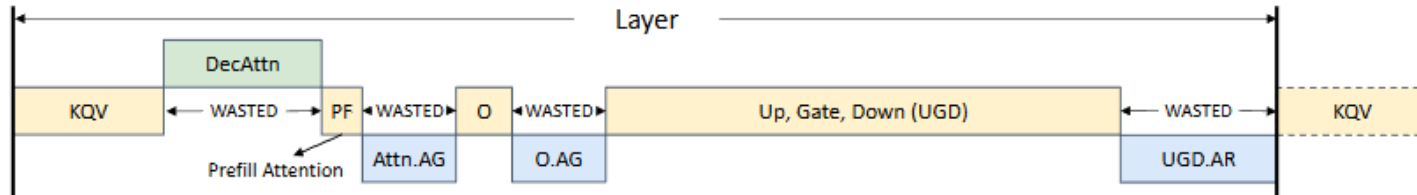
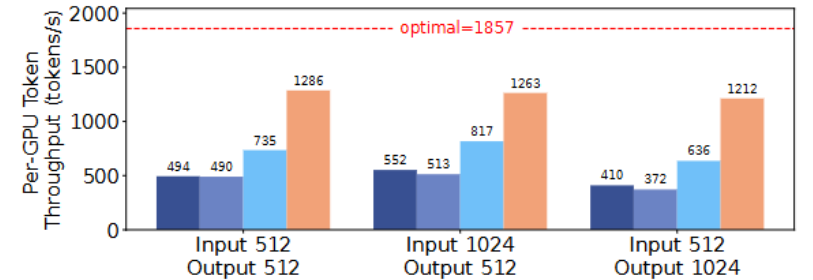
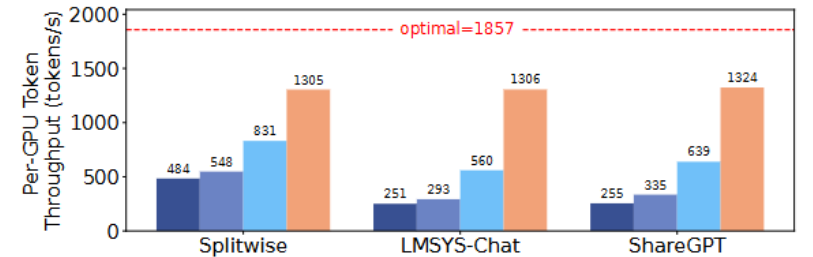


Figure 4: Execution pipeline of existing systems. The green, yellow, and blue operations correspond to memory-, compute-, and network-bound operations. Operations in the previous and next layer are denoted by dotted borders. "WASTED" shows the stages in the pipeline where the most constrained resource, compute, is underutilized. Small operations (i.e. layernorm, activation, etc.) are omitted for simplicity.

Legend: vLLM (dark blue), DeepSpeed-FastGen (medium blue), TensorRT-LLM (light blue), NanoFlow (Ours) (orange)



(a) LLaMA-2-70B, 8 GPU, TP=8, Constant Input & Output Length

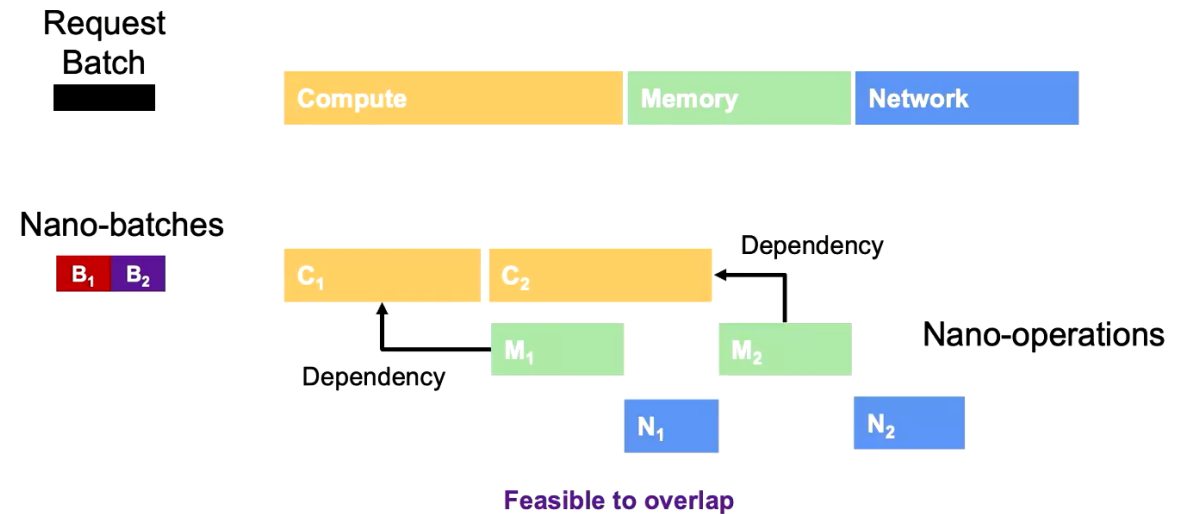


(b) LLaMA-2-70B, 8 GPU, TP=8, Input & Output Length from Dataset

Figure 7: Offline throughput comparison. NanoFlow outperforms all baselines for all the workload settings. TP stands for the number of GPUs used with tensor parallelism.

Intra-device Parallelism

- ▶ 큰 batch
 - ▶ 장점: 모델 가중치를 불러오는 횟수가 적음
- ▶ Nano batch
 - ▶ 앞선 증명으로 compute bound한 작업들에 대한 활용도를 증가시킨다면 작은 batch를 사용하는 것이 정당화될 수 있음
 - ▶ 디바이스 내부 병렬화 사용



Design

Automated Pipeline Search

- ▶ Kernel Profiling and Interference Modeling
 - ▶ Determining the maximum dense batch size
 - ▶ Profiling interference-free kernels
 - ▶ Profiling kernels with interference

Table 3: Performance P of GEMV and network kernels with different resource utilization R .

Operations	Resource Utilization (R)						
	0	0.1	0.2	...	0.8	0.9	1
GEMM (by definition)	0	0.1	0.2	...	0.8	0.9	1
GEMV	0	0.2	0.3	...	0.85	0.95	1
Network	0	0.3	0.5	...	0.9	1	1

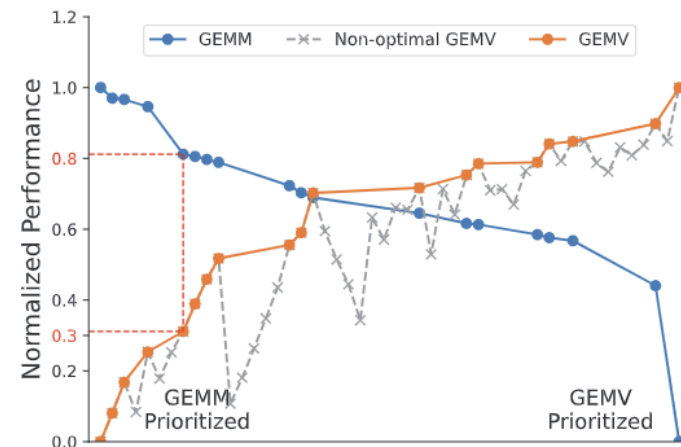
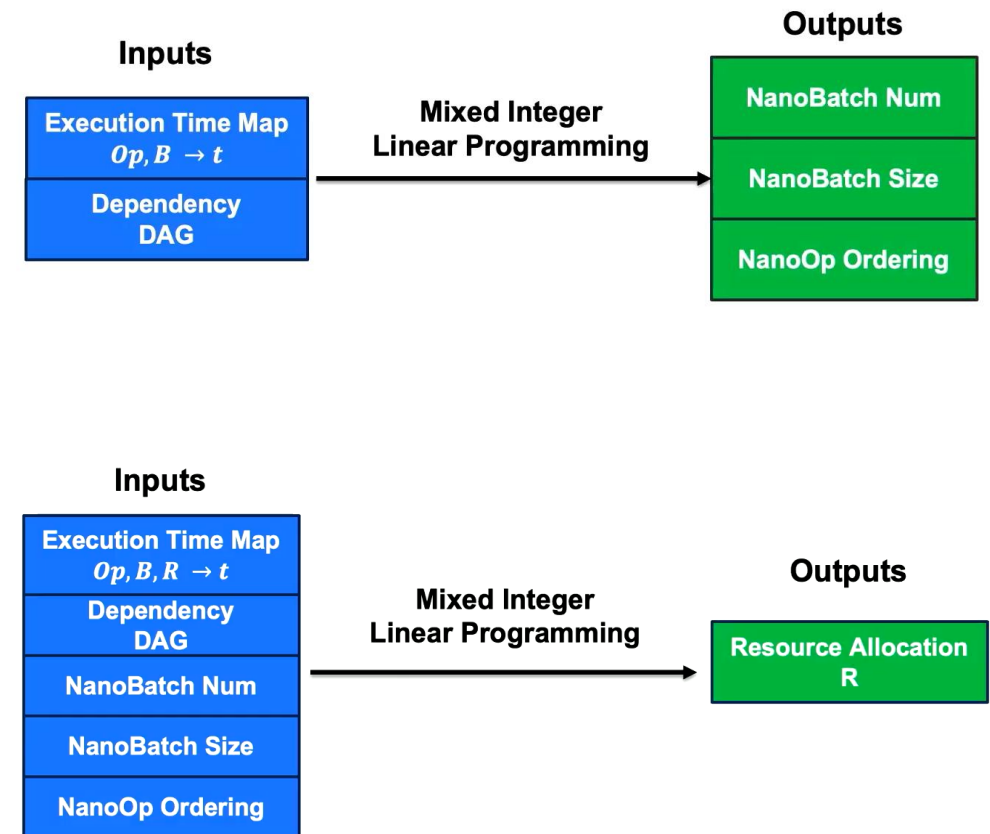


Figure 5: Interference characteristics between GEMM and GEMV kernels. The points on the x-axis correspond unique GEMM-GEMV implementation pairs. The y-axis denotes the GEMM and GEMV kernels' normalized performance P .

Automated Pipeline Search

- ▶ Auto-search Stage I: Pipeline Structure Search
 - ▶ Constraints on the number of nano-operations
 - ▶ Constraints on batch sizes and execution times
 - ▶ Constraints on dependencies
 - ▶ Constraints on overlapping
 - ▶ Constraints on operation transformations
- ▶ Auto-search Stage II: Refining the Pipeline
 - ▶ Interference 고려



Example Pipelines

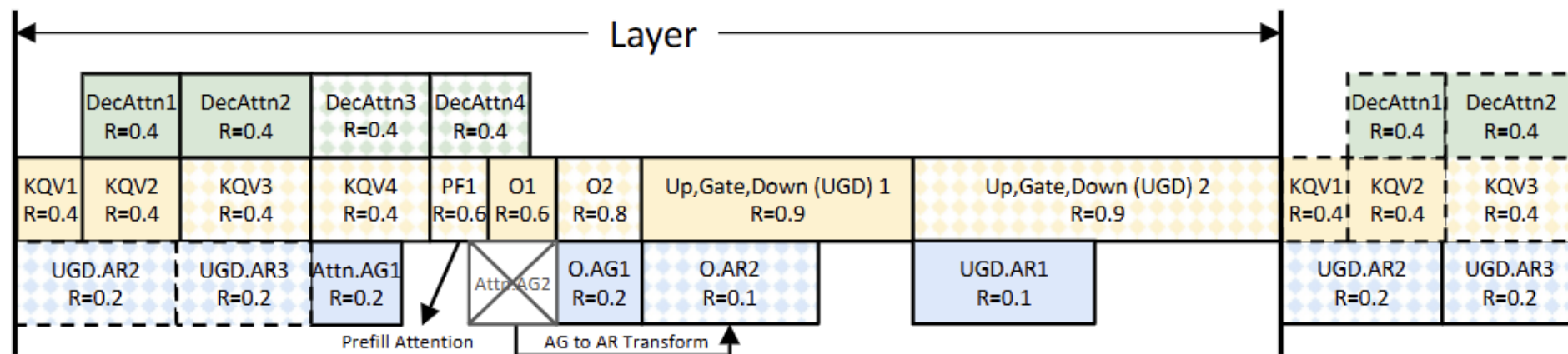


Figure 6: Execution pipeline of LLaMA-2 70B, automatically generated by NanoFlow. The solid background and shaded background represents input batch 0-768 and 768-2048, respectively. R stands for resource utilization. By overlapping the compute-, memory-, and network-intensive operations, NanoFlow increases compute utilization and improves the serving throughput.

NanoFlow Runtime

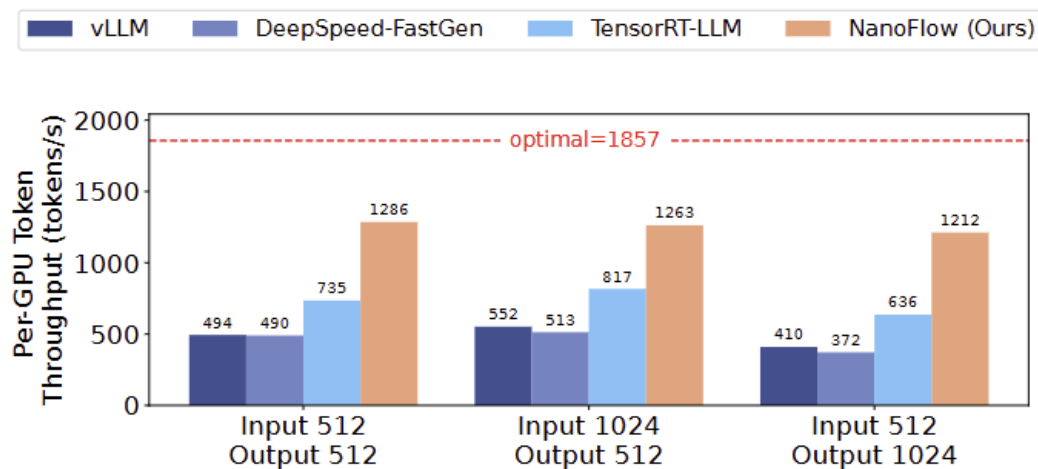
- ▶ Request Scheduling
 - ▶ Batch formation
 - ▶ Asynchronous scheduling
- ▶ KV-cache Management
 - ▶ Simultaneous offloading
 - ▶ Host KV-cache management
 - ▶ KV-cache loading and scattering

Evaluation

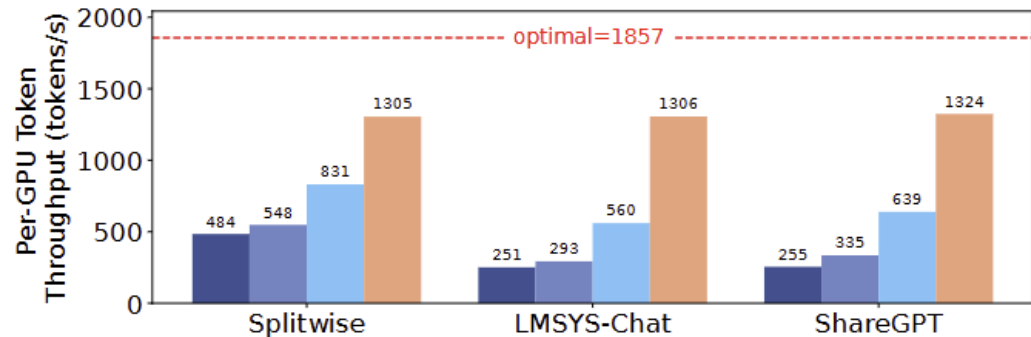
Throughput

Dataset	Avg. Input (Std)	Avg. Output (Std)
Splitwise [32]	1155 (1109)	211 (163)
LMSYS-Chat [56]	102 (169)	222 (210)
ShareGPT [1]	246 (547)	322 (244)

Table 4: The average and standard deviation of input and output lengths in the sampled datasets.



(a) LLaMA-2-70B, 8 GPU, TP=8, Constant Input & Output Length



(b) LLaMA-2-70B, 8 GPU, TP=8, Input & Output Length from Dataset

Figure 7: Offline throughput comparison. NanoFlow outperforms all baselines for all the workload settings. TP stands for the number of GPUs used with tensor parallelism.

Latency

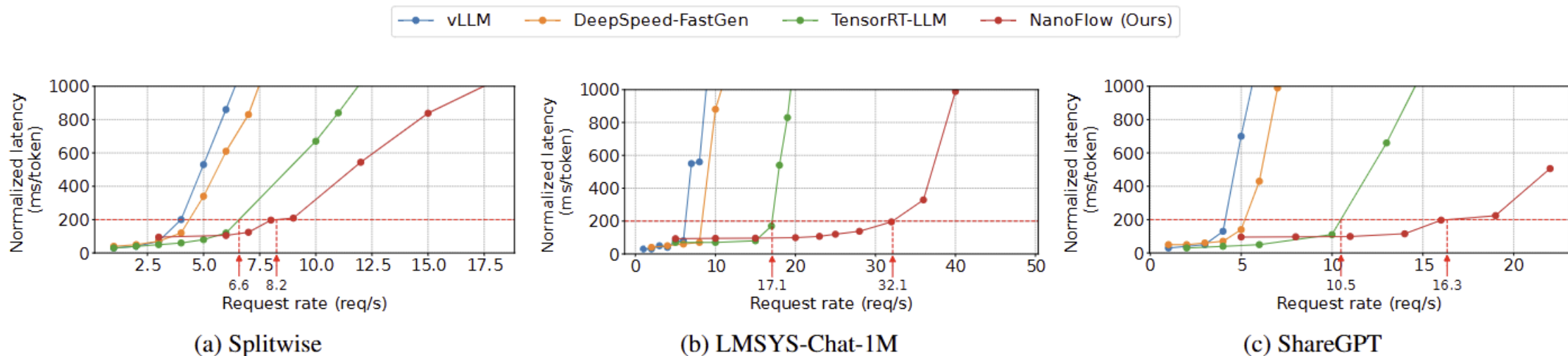


Figure 8: Latency comparison. The x-axis shows the number of incoming requests per second, and the y-axis shows the normalized latency. NanoFlow handles higher request rates within 200ms SLO constraints.

Ablation Study

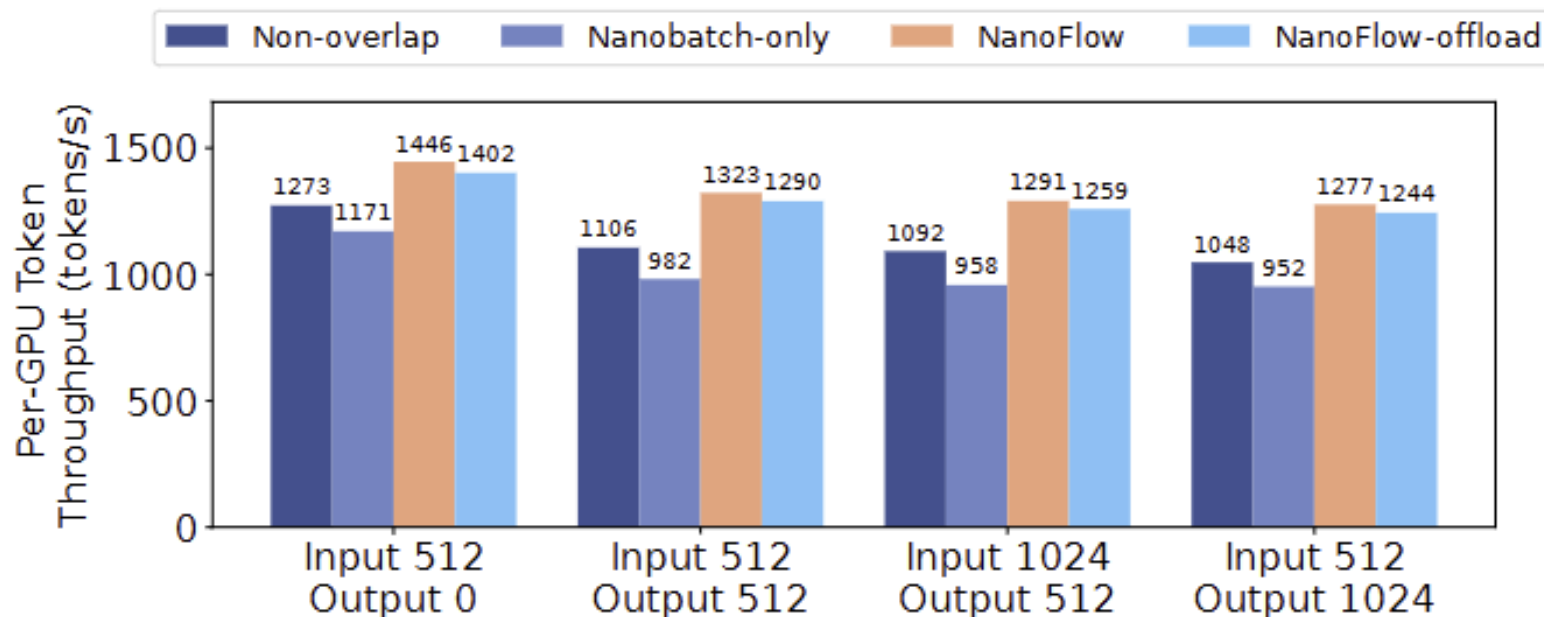
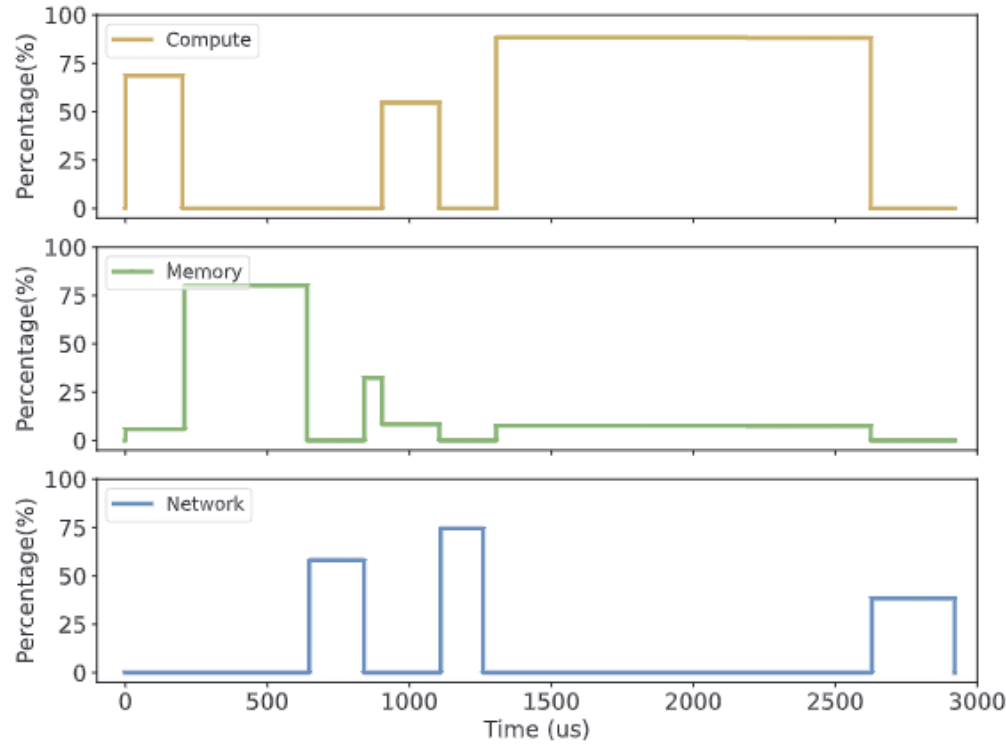
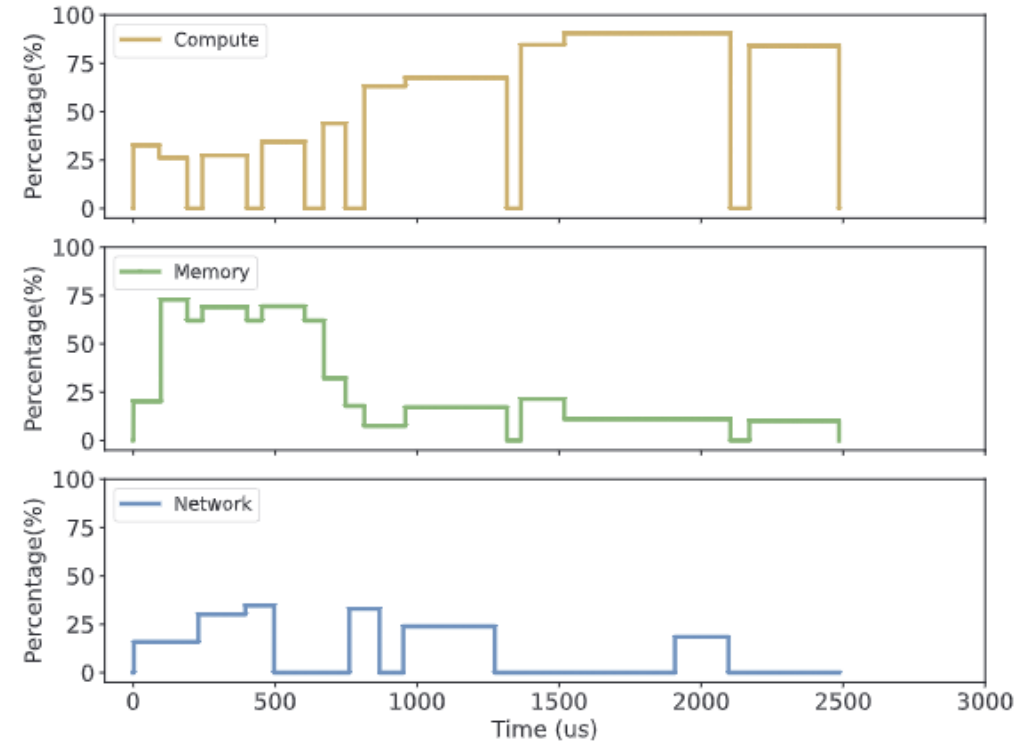


Figure 9: Ablation study results for NanoFlow. Nano-batching and overlapping improves NanoFlow's performance.

Resource Usage



(a) Non-overlap pipeline resource usage



(b) NanoFlow resource usage

Figure 10: The non-overlapping pipeline and NanoFlow's resource usage during inference of a single layer. NanoFlow achieves high compute usage across the whole pipeline by simultaneously utilizing memory and network bandwidth.

Performance on Other LLMs

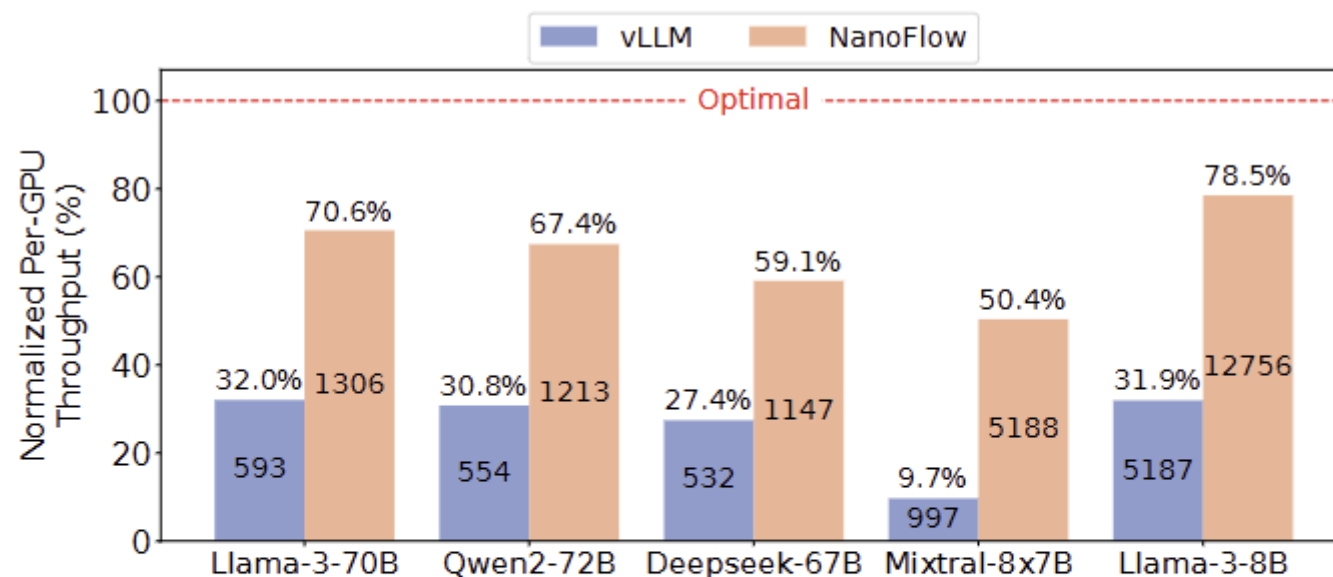


Figure 11: Performance of NanoFlow instances in terms of tokens per second per GPU for other models. Compared with vLLM, NanoFlow significantly increases throughput. NanoFlow achieves up to 78.5% of optimal throughput.

Conclusion

- ▶ reveal the compute-bound nature of modern LLM serving workloads.
- ▶ propose NanoFlow, a novel end-to-end LLM serving system
- ▶ $1.91\times$ throughput improvement over the SOTA system